

BE : EN GÉNÉRALISANT L'ALGORITHME DU GRADIENT CONJUGUÉ...

Consignes Générales :

- Par groupe de 5 à 6 étudiants maximum, vous devez rédiger un rapport dans lequel vous répondrez aux questions du sujet et plus globalement au problème demandé. La qualité de rédaction ainsi que tous les compléments que vous apporterez (réflexions, analyses des réponses, tests sur d'autres données, etc...) seront fortement pris en compte. Ce rapport devra être rédigé sous un traitement de texte (les pages manuscrites ne sont pas acceptées) et rendu sous la forme d'un fichier .pdf.
- Toutes les fonctions programmées le seront en Python et seront abondamment commentées. Le rendu du projet se fera sous la forme d'un fichier .zip contenant le rapport ainsi que l'ensemble des programmes et données nécessaires à l'exécution des programmes.
- Les fonctions programmées doivent l'être dans un fichier librairie ayant pour nom : **Ma316_BE_lib.py**. Dans ce fichier les différentes fonctions doivent être regroupées dans des blocs (commande : #%%) correspondants aux parties du sujet.
- Les applications numériques des fonctions du sujet seront dans un fichier ayant pour nom : **Ma316_BE_main.py**. Dans ce fichier les différents cas d'étude doivent être regroupés dans des blocs séparés.
- Les différentes données utilisées devront être enregistrées dans un dossier ayant pour nom **Data**. Ce dossier doit être mis dans le même répertoire que vos fichiers **Ma316_BE_lib.py** et **Ma316_BE_main.py**.
- Les noms des différents membres du groupe doivent apparaître en commentaire en début de fichier.
- Le non-respect des consignes précédentes entraînera des points de pénalité pouvant être conséquents...
- Concernant Python, le rendu attendu est l'utilisation des fonctions de bases des librairies Numpy, Matplotlib et OpenCV. Néanmoins l'utilisation de fonctions plus avancées qui améliorent les rendus graphiques par exemple et/ou la programmation en terme de classe seront bien évidemment acceptées et seront valorisées.
- Les questions d'analyse de résultats sont généralement ouvertes. La qualité de vos réflexions sera fortement prise en compte pour la notation de ces questions.
- Il est impératif de lire le sujet en entier.
- Enfin on rappelle qu'un BE est un examen et comme tout examen il arrive de ne pas pouvoir répondre à toutes les questions.

Dans ce BE nous allons étendre/généraliser deux notions étudiées dans ce module : les équations d'Euler-Lagrange et l'algorithme du gradient conjugué. Ces deux points feront l'objet des deux premières parties et peuvent être traités de manière globalement séparées. Enfin dans une troisième et dernière partie nous utiliserons les résultats des deux premières parties afin d'obtenir des algorithmes efficaces pour résoudre des problèmes en électrostatique et dans un second temps nous utiliserons ces mêmes équations (à des variations près) afin de réaliser du traitement d'image. Ces dernières équations font bien entendu sens en physique (électrostatique, gravitation, etc...) mais les contextes physiques s'y rapportant étant assez long et complexe à introduire nous avons fait le choix du « visuel » rapide à introduire et de l'application de ces équations au traitement d'image.



FIGURE 1 – Exemple de sortie possible de l'algorithme final de ce BE : **à gauche**, une image que l'on va modifier, ici le tableau *La Clairvoyance* de René Magritte (1936) , **au centre** : une image que nous allons utiliser afin de « customiser » le tableau de Magritte, **à droite** : le résultat final de l'algorithme où l'on a « injecté » l'aigle dans le tableau à la place de l'oiseau originel.

Avant de débiter ce sujet on rappelle quelques notions de Python concernant la modification d'image (on pourra trouver plus de détail dans le polycopié de cours de Ma223 : Moodle/Teams).

Sous Python il existe un grand nombre de librairie permettant de réaliser du traitement d'image. Par exemple la librairie Matplotlib permet d'afficher des images et également de les charger en utilisant la librairie PIL. On renvoie pour l'utilisation de ces librairies à la page : [tutorial matplotlib image](#). Nous allons ici surtout nous concentrer sur la librairie OpenCV et voir comment il est possible de manipuler des images avec Python en utilisant cette librairie. Pour installer OpenCV, parmi les multiples façon de procéder on pourra ouvrir Spyder et taper la commande suivante directement dans la console ou dans le prompt d'Anaconda :

```
pip install opencv-python
```

ou bien :

```
conda install -c conda-forge opencv
```

On rappelle que l'on peut importer une image (format *jpg* ou *png*) grâce à Python sous forme de matrices (très grandes matrices en général) et les quelques points suivants :

- On commence par enregistrer l'image à importer dans le dossier où se trouve votre fichier python (il est conseillé de ne pas prendre une image de trop grande qualité pour la santé de votre ordinateur...). Par exemple l'image d'une fraise dont le nom de fichier est : *fraise.jpg* .
- On importe alors l'image grâce à la librairie OpenCV :

```
import cv2
image=cv2.imread('fraise.jpg')
```

image est composée de trois matrices, chacune représentant les intensités des couleurs primaires BGR (bleu, vert, rouge) lorsque l'on utilise la fonction `imread` d'OpenCV. Une image « standard » i.e. couleur est composée de trois matrices, chacune représentant les intensités des couleurs primaires BGR (bleu, vert, rouge) lorsque l'on utilise la fonction `imread` d'OpenCV et RGB (rouge, vert, bleu) avec PIL et Matplotlib. Vous pouvez voir dans l'explorateur de variable qu'une image est en réalité la donnée de trois matrices : une matrice pour l'intensité du rouge, une pour le vert et une couche pour le bleu. Chacune de ces intensités est codée par un « entier » (type `uint8`) compris entre 0 et 255 (i.e. sur 2^8 bits). Les images en nuances de gris possèdent une unique couche et sont donc des matrices. **Attention les images étant des array de type `uint8` il faudra absolument penser à convertir vos images en matrice de flottants lorsque vous voulez réaliser des opérations mathématiques dessus (sous entendu avec des réels) et réciproquement les reconverter en `uint8` lorsque vous voulez les afficher.**

- On peut soit travailler avec la matrice « triple » *image* soit si la couleur nous importe peu convertir l'image en niveau de gris. Pour cela on peut faire la moyenne des intensités des trois couches de couleurs « primaires » pour obtenir une seule matrice :

```
M = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

ou :

```
M=cv2.imread('fraise.jpg',0)
```

La matrice *M* est alors une version matricielle de notre image importée. Une commande alternative est la suivante : qui importe directement l'image en niveau de gris.

- On peut afficher l'image en niveaux de gris en utilisant la commande suivante :

```
cv2.imshow('fraise',M)
```

Les niveaux de gris sont incarnés par des nombres de 0 à 255.

- On peut également enregistrer la matrice sous forme d'un fichier image (ici *jpg*) :

```
cv2.imwrite('fraise_gris.jpg', M)
```

- Attention il faudra absolument penser à convertir vos images en matrice de flottant lorsque vous voulez réaliser des opérations mathématiques. Observer la différence entre les deux scripts suivantes :

(a) Version '`uint8`' :

```
a=np.array([0,255]).astype('uint8')
print(a+a)
```

(b) Version '`float`' :

```
a=np.array([0,255])
print(a+a)
```

1 Équation d'Euler-Lagrange à deux variables

Dans cette partie nous allons étendre (très légèrement...on pourrait faire bien mieux...) les calculs effectués lors du TD1 de ce module à des actions données par des intégrales doubles (et non plus uniquement d'une seule variable que nous avons notée t). En effet en TD, nous avons résolu le problème de la recherche des points critiques d'une fonctionnelle d'action donnée par un lagrangien. Nous avons interprété dans les cadres physiques classiques la forme de tels lagrangiens comme la formalisation des « contraintes » qui « contraignent » la trajectoire de notre système physique étudié dans son espace de configuration (=ses degrés de liberté). Ces contraintes étant supposés décrites localement (temporellement parlant) par le lagrangien, l'action incarne la somme (sur le temps) de toutes ces contraintes. Dans cette partie nous allons dépasser ce cadre restreint à la seule variable temporelle et nous considérerons des actions données par des intégrales doubles. Plaçons nous dans le cadre variationnel suivant.

- $\Omega \subset \mathbb{R}^2$ est un domaine fermé et borné du plan. On supposera que la frontière $\partial\Omega$ ($:=$ le bord) de Ω est décrit par une courbe paramétrique orientée $\gamma : [0, 1] \rightarrow \partial\Omega \subset \mathbb{R}^2$ de classe $\mathcal{C}^2$. Cette courbe formera un lacet i.e. une courbe paramétrique fermée où $\gamma(0) = \gamma(1)$. On notera x, y les variables de Ω .
- Nous prendrons comme espace des configurations : $C = \mathbb{R}$. Il n'y aura donc qu'un seul degré de liberté ici.
- On prendra pour espace des historiques : $H := \mathcal{C}^2(\Omega, C)$.

On notera un élément de H par : $u : \Omega \rightarrow C$ et l'on notera $\partial_x u := \frac{\partial u}{\partial x}$ et $\partial_y u := \frac{\partial u}{\partial y}$ les dérivées partielles par rapport à la première respectivement seconde variable de Ω .

On considérera l'action lagrangienne à deux variables de la forme :

$$\boxed{\begin{array}{lcl} S : & H & \rightarrow \mathbb{R} \\ & u & \mapsto S(u) := \int \int_{\Omega} L(u, \partial_x u, \partial_y u, x, y) \, dx dy \end{array}}$$

où L est une fonction deux fois différentiables : $L : C \times C \times C \times \Omega \rightarrow \mathbb{R}$ que l'on appellera encore lagrangien de l'action S .

1. Fixons $v : \Omega \rightarrow C \in H$. Démontrer que la dérivée directionnelle de l'action S dans la direction v s'écrit sous la forme :

$$D_v S(u) = \int \int_{\Omega} \left(\frac{\partial L}{\partial u} v + \frac{\partial L}{\partial (\partial_x u)} \partial_x v + \frac{\partial L}{\partial (\partial_y u)} \partial_y v \right) dx dy$$

2. Notons la forme différentielle :

$$dw := -P(x, y) dx + Q(x, y) dy$$

où :

$$\begin{cases} P(x, y) := v(x, y) \frac{\partial L}{\partial (\partial_y u)}(u(x, y), \partial_x u(x, y), \partial_y u(x, y), x, y) \\ Q(x, y) := v(x, y) \frac{\partial L}{\partial (\partial_x u)}(u(x, y), \partial_x u(x, y), \partial_y u(x, y), x, y) \end{cases}$$

En utilisant les notations précédentes rappeler la formule de Green-Riemann associée à l'intégration de la forme différentielle dw sur $\partial\Omega$. En déduire une formule d'intégration par parties à deux variables.

3. On suppose que $v(x, y) = 0$ pour tout $(x, y) \in \partial\Omega$. Déduire de la question précédente et du lemme fondamental du calcul variationnel que les points critiques u de S vérifient sur Ω :

$$\boxed{\frac{\partial L}{\partial u} = \frac{d}{dx} \left(\frac{\partial L}{\partial (\partial_x u)} \right) + \frac{d}{dy} \left(\frac{\partial L}{\partial (\partial_y u)} \right)}$$

On obtient ainsi les **équations d'Euler-Lagrange** à deux variables que nous cherchions.

4. **Bonus** : Pouvez-vous généraliser cette formule aux cas de plus de 2 variables ? puis au cas où u est une fonction à valeurs dans $\mathbb{R}^N$?
5. **Application** : On considère une corde de longueur L dans un repère orthonormé (e_x, e_y) et on note : $u(x, t)$ la hauteur sur l'axe e_y en fonction de la position x et du temps t . On posera : $\Omega = [t_0, t_f] \times [0, L]$ et on supposera que u est une fonction de classe $\mathcal{C}^2$. Le lagrangien étant l'expression locale des contraintes agissantes sur la « trajectoire » dans l'espace des degrés de libertés de notre système physique. Nous supposons ici qu'il est de la forme :

$$L(u, \partial_x u, \partial_t u, x, t) := \frac{1}{2} \rho \partial_t u(x, t)^2 - f(\partial_x u(x, t))$$

où ρ est la densité linéique de la corde et $f : \mathbb{R} \rightarrow \mathbb{R}$ est une fonction que nous supposons de classe $\mathcal{C}^2$.

- (a) (question de physique) Que représente chacune des deux parties de ce lagrangien ?
- (b) En utilisant la question précédente montrer que le mouvement de la corde doit vérifier l'équation aux dérivées partielles suivantes :

$$\rho \partial_{t,t} u(x, t) - f''(\partial_x u(x, t)) \partial_{x,x} u(x, t) = 0$$

où l'on a noté :

$$\partial_{t,t} u(x, t) := \frac{\partial^2 u}{\partial t^2}(x, t) \quad \text{et} \quad \partial_{x,x} u(x, t) := \frac{\partial^2 u}{\partial x^2}(x, t)$$

Quelle hypothèse peut-on faire sur f pour retrouver l'équation de d'Alembert :

$$\partial_{t,t} u(x, t) - c^2 \partial_{x,x} u(x, t) = 0$$

où c est la vitesse de propagation de l'onde dans le milieu considéré ? Comment interpréter physiquement cette hypothèse (et par suite l'influence physique de f) ?

- (c) Soit s une fonction définie sur Ω à valeurs réelles. Quel lagrangien poser pour obtenir l'équation des ondes avec source suivante :

$$\rho \partial_{t,t} u(x, t) - f''(\partial_x u(x, t)) \partial_{x,x} u(x, t) = s(x, t)$$

Interpréter physiquement cette contrainte.

2 Une généralisation du gradient conjugué

On se placera dans toute cette partie sur un espace vectoriel de dimension fini E muni d'un produit scalaire, $\langle \bullet, \bullet \rangle$ et de la norme associée $\|\bullet\|$. Nous dirons qu'une application linéaire $A : E \rightarrow E$ est **symétrique** si pour tout $(x, y) \in E^2$:

$$\langle x, A(y) \rangle = \langle A(x), y \rangle$$

et nous dirons que A est **semi-définie positive** si $x \in E$:

$$\langle x, A(x) \rangle \geq 0$$

Nous dirons qu'elle est **définie positive** si l'inégalité est stricte pour tout $x \in E \setminus \{0_E\}$.

On définit une fonction quadratique sur $(E, \langle \bullet, \bullet \rangle)$ par une fonction de la forme pour $x \in E$:

$$J(x) := \frac{1}{2} \langle x, A(x) \rangle - \langle b, x \rangle$$

où A est une application linéaire symétrique, $b \in E$.

1. Commençons par un résultat général simple. Montrer que la somme de deux fonctions convexes sur E à valeurs dans $\mathbb{R}$ est encore une fonction convexe.
2. Notons : $f : x \mapsto \langle x, A(x) \rangle$ où A est une application linéaire symétrique. Montrer que f est convexe si et seulement si A est semi-définie positive et strictement convexe si et seulement si A est définie positive.
3. Dédurre des questions précédentes un critère de convexité pour J et un critère de stricte convexité.
4. **On supposera à présent et dans les questions suivantes que J est une fonction quadratique convexe.** Nous allons montrer que si J admet un minium local en x alors c'est un minimum global. Être un minimum local signifie qu'il existe $R > 0$ tel que :

$$J(x) = \min \{ J(z) \mid \forall z \in E, \|z - x\| \leq R \}$$

Nous allons raisonner par l'absurde. Montrer que s'il existe un y tel que $\|y - x\| > R$ et $J(y) < J(x)$, on peut trouver un z de la forme :

$$z = (1 - \theta)x + \theta y$$

pour un $\theta \in [0, 1]$ bien choisi tel que :

$$\|z - x\| \leq R \quad \text{et} \quad J(z) < J(x)$$

5. Fixons $h \in E$, montrer que pour tout $x \in E$, la dérivée directionnelle de J en x dans la direction h s'écrit :

$$D_h J(x) = \langle h, A(x) - b \rangle$$

On appellera gradient de J la fonction notée ∇J et définie pour tout $x \in E$ par :

$$\boxed{\nabla J(x) := A(x) - b}$$

6. Montrer que J est convexe si et seulement si pour tout $(x, y) \in E \times E$:

$$J(y) \geq J(x) + \langle y - x, \nabla J(x) \rangle$$

7. En déduire que x est minimum global pour J si et seulement si : $\nabla J(x) = 0$.
 8. En utilisant le cours et l'exercice du TD3 sur le gradient conjugué, écrivez sous la forme d'un algorithme une version générale de l'algorithme du gradient conjugué valable dans un espace vectoriel de dimension fini E muni d'un produit scalaire $\langle \bullet, \bullet \rangle$ et pour une fonction J quadratique convexe.
 9. En notant $(x_{(k)})_{k \in \mathbb{N}}$ la suite des itérés de l'algorithme du gradient conjugué précédent donner une expression pour un $k \in \mathbb{N}$ de l'écart théorique entre deux itérés :

$$e_{(k)} := J(x_{(k+1)}) - J(x_{(k)})$$

D'après la formule précédente, quel(s) critère(s) d'arrêt peut-on demander ?

10. Rédiger une fonction Python : **grad_conj(ps,A,b,x0,epsilon,itermax)** qui prend en entrée un produit scalaire **ps**, une application linéaire **A**, deux « vecteurs » **b** et **x0**, une tolérance (un réel positif) **epsilon**, **itermax** un entier/nombre d'itération maximal et qui réalise l'algorithme du gradient conjugué de la question précédente.
 11. **Application** : Nous allons nous intéresser à un cas particulier de l'**équation de Sylvester**. On retrouve cette équation dans de nombreux problèmes (du traitement d'image à l'étude de système dynamique linéaire, etc...). Soient P, Q et R des matrices de tailles respectives (m, m) , (n, n) et (m, n) et l'on cherche $X \in \mathcal{M}_{m,n}(\mathbb{R})$ tel que :

$$(\mathcal{S}) \quad PX + XQ = R$$

En reprenant les notations de cette partie, on pose : $E = \mathcal{M}_{m,n}(\mathbb{R})$, $\langle \bullet, \bullet \rangle = \text{Trace}(\bullet^T \bullet)$, $A(X) := PX + XQ$ et :

$$J(X) := \frac{1}{2} \langle X, A(X) \rangle - \langle R, X \rangle$$

- (a) Démontrer que J est une fonction quadratique sur $(E, \langle \bullet, \bullet \rangle)$.
 (b) Justifier que si P et Q sont des matrices symétriques semi-définies positives alors J admet un minimum (non nécessairement unique).
 (c) En déduire que les minima de J vérifient l'équation de Sylvester $(\mathcal{S})$.
 (d) Tester votre algorithme avec les matrices :

$$P = Q := \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \quad R := 0_{2,2}$$

Vous comparerez le résultat de la résolution « à la main » avec celui produit par votre programme.

3 En mélangeant tout cela...

Dans cette partie nous allons appliquer les résultats des deux parties précédentes en utilisant des variations autour de l'équation de Poisson.

3.1 Un problème d'électrostatique

Soit $\Omega \subset]0, L_1[\times]0, L_2[\subset \mathbb{R}^2$ un domaine, on s'intéresse au potentiel électrostatique V sur Ω en fonction d'une densité surfacique de charge ρ . **On supposera que V est nul sur $\partial\Omega$** i.e. que le potentiel électrostatique est nul « loin » de la répartition des charges. En notant classiquement ϵ_0 la permittivité du vide nous supposons que le potentiel électrostatique est un point critique de l'action suivante :

$$S(V) := \int \int_{\Omega} \left(\frac{1}{2} \|\nabla V(x, y)\|_2^2 - \frac{1}{\epsilon_0} \rho(x, y) V(x, y) \right) dx dy$$

1. En utilisant les résultats de la partie 1, montrer qu'un point critique de l'action précédente vérifie l'équation de Poisson suivante :

$$(\mathcal{E}) \quad \Delta V(x, y) = -\frac{1}{\epsilon_0} \rho(x, y), \quad \forall (x, y) \in \Omega$$

où Δ note l'opérateur de Laplace.

2. Nous allons discrétiser le problème précédent. On fixe M et N deux entiers non nuls et on pose la discrétisation suivante :

$$\Delta x := \frac{L_1}{M+1} \quad \Delta y := \frac{L_2}{N+1} \quad \sim \begin{cases} x_i := i\Delta x & i \in \llbracket 0, M+1 \rrbracket \\ y_j := j\Delta y & j \in \llbracket 0, N+1 \rrbracket \end{cases}$$

Comme V est nul sur $\partial\Omega$ on supposera que :

$$\begin{cases} V(0, y) = V(L_1, y) = 0 & \forall y \in \Omega \\ V(x, 0) = V(x, L_2) = 0 & \forall x \in \Omega \end{cases}$$

cela nous permettra d'identifier le potentiel recherché V sur Ω à une matrice $\mathcal{M}_{M,N}(\mathbb{R})$ dont les coefficients sont données par :

$$V_{i,j} := V(x_i, y_j)$$

- (a) Soit $f : I \subset \mathbb{R} \rightarrow \mathbb{R}$ une fonction quatre fois dérivable et notons : $x_i := ih \in I$ avec $h \in \mathbb{R}$ considéré comme « petit », une suite de points de I . On supposera que $i \in \llbracket 0, N \rrbracket$. On notera : $f_i := f(x_i)$. La méthode des différences finies d'ordre 2 permet d'approximer la dérivée seconde d'une fonction (à l'ordre 2 si cette fonction est $\mathcal{C}^4$) par la formule :

$$\frac{d^2 f}{dx^2}(x_i) \simeq \frac{f_{i+1} - 2f_i + f_{i-1}}{h^2}$$

Montrer que cette formule est obtenue par la somme des deux développements de Taylor :

- $f(x_{i+1}) = f(x_i + h) = f(x_i) + f'(x_i)h + \frac{1}{2}f''(x_i)h^2 + \frac{1}{3!}f'''(x_i)h^3 + O(h^4)$
- $f(x_{i-1}) = f(x_i - h) = f(x_i) - f'(x_i)h + \frac{1}{2}f''(x_i)h^2 - \frac{1}{3!}f'''(x_i)h^3 + O(h^4)$

- (b) Montrer brièvement à l'aide de la question précédente pourquoi le problème $(\mathcal{E})$ peut s'approximer par les équations suivantes pour tout $(i, j) \in \llbracket 1, M \rrbracket \times \llbracket 1, N \rrbracket$:

$$\frac{V_{i+1,j} - 2V_{i,j} + V_{i-1,j}}{(\Delta x)^2} + \frac{V_{i,j+1} - 2V_{i,j} + V_{i,j-1}}{(\Delta y)^2} = -\frac{1}{\epsilon_0} \rho(x_i, y_j)$$

- (c) En déduire qu'une forme matricielle approchée de $(\mathcal{E})$ s'écrit de la manière suivante où L_M et L_N sont des matrices carrées à déterminer :

$$(\mathcal{E}_d) \quad \left(\frac{1}{(\Delta x)^2} L_M \right) V + V \left(\frac{1}{(\Delta y)^2} L_N \right) = \frac{1}{\epsilon_0} R$$

où $R \in \mathcal{M}_{M,N}(\mathbb{R})$ est la matrice dont les coefficients sont données par :

$$R_{i,j} := \rho(x_i, y_j)$$

- (d) **Applications numériques :** En utilisant la partie précédente sur la généralisation du gradient conjugué rédiger un programme python qui résout le problème précédent ($\mathcal{E}_d$) à l'aide d'un gradient conjugué matriciel. Pour simplifier les applications numériques on supposera que $\Delta x = \Delta y = 1$ et $\epsilon_0 = 1$. On prendra pour R pour commencer la matrice $R.npy$ puis vous testerez avec d'autres matrices en entrée. Cette matrice représente la vue en coupe de la répartition des charges sur les deux plaques d'un condensateur.
- (e) Rappeler l'expression du champs électrique E en fonction du potentiel électrostatique V . Justifier que l'on peut approximer les dérivées partielles de V à l'ordre 2 par les formules suivantes :

$$\frac{\partial V}{\partial x}(x_i, y_j) \simeq \frac{V_{i+1,j} - V_{i-1,j}}{2\Delta x} \quad \text{et} \quad \frac{\partial V}{\partial y}(x_i, y_j) \simeq \frac{V_{i,j+1} - V_{i,j-1}}{2\Delta y}$$

Comme dans les questions précédentes en déduire qu'il existe des matrices carrées D_M et D_N telles que le champ électrique s'écrit de manière approchée :

$$E \simeq (D_M V, V D_N^T)$$

Tracer le résultat obtenu. On pourra utiliser les commandes de matplotlib : **contourf** (pour les lignes de niveaux) et **streamplot** (pour les lignes de champs).

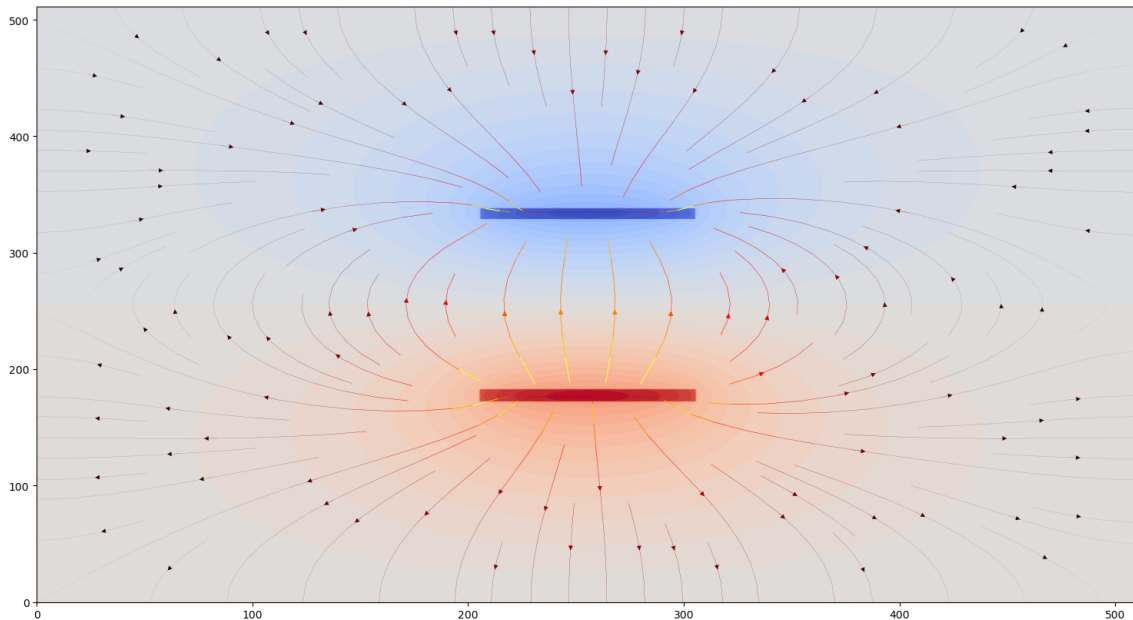


FIGURE 2 – Exemple de représentation possible du champ électrique pour l'exemple du condensateur (deux plaques chargées de charges opposées) i.e. pour le calcul effectué sur la matrice donnée $R.npy$. Le fond coloré représente les valeurs du champ de potentiel électrostatique V et les flèches représentent le champ électrique E associé.

- En utilisant la librairie OpenCV réaliser une fonction qui permet de dessiner grâce à la souris la répartition des charges positives (d'une valeur fixée, par exemple 1) et la répartition des charges négatives (d'une valeur fixée également, -1 par exemple). Cette fonction renverra une matrice de taille M, N que l'on utilisera comme matrice d'entrée R (matrice de la densité surfacique de la répartition des charges). Tester différentes configurations et commenter les potentiels électrostatiques V obtenues après résolutions des équations ($\mathcal{E}_d$) associées.

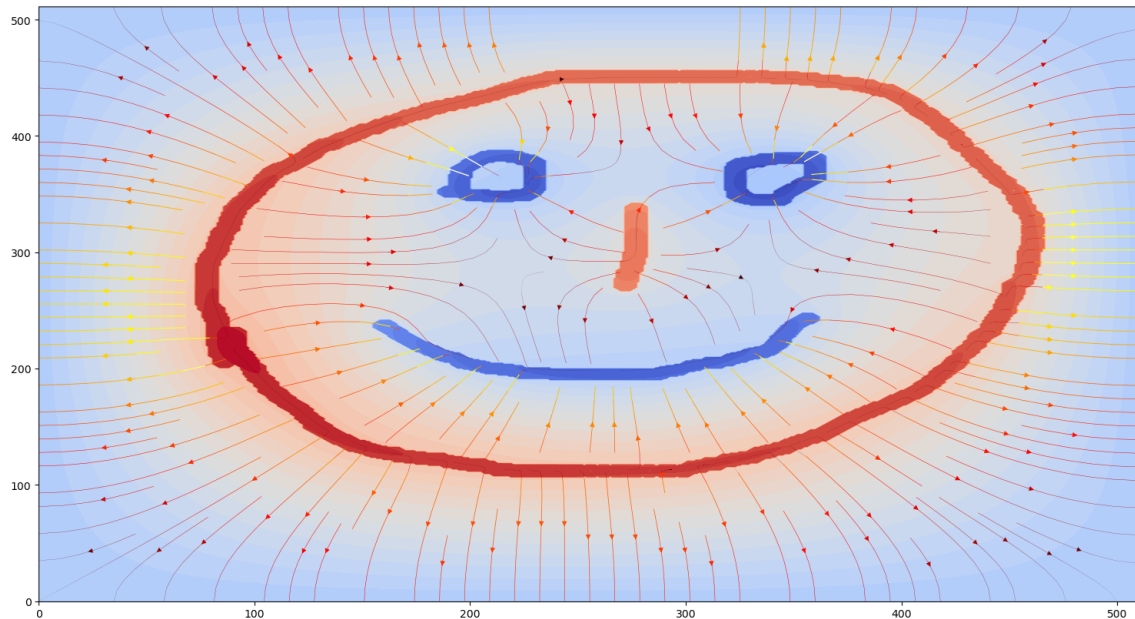


FIGURE 3 – Exemple de sortie : champ électrique fictif produit par une densité surfacique dessinée à la main avec la fonction précédente. En rouge les charges + et en bleu les charges -. Les courbes fléchées représentent le champ électrique produit.

Pour les callback avec la souris on pourra utiliser les commandes fournis dans la documentation d'OpenCV : `MouseEventTypes`. Un début de code possible afin de « sélectionner » des points grâce à la souris est donnée par :

```
drawingp, drawingm = False, False # true if mouse is pressed
ix, iy = -1, -1

def draw_curve(event, x, y, flags, param):
    global ix, iy, drawingp, drawingm, img

    if event == cv2.EVENT_LBUTTONDOWN:
        drawingp = True

    elif event == cv2.EVENT_MOUSEMOVE :
        if drawingp == True :
            cv2.circle(img, (x, y), 3, (0, 0, 255), 4)
    elif event == cv2.EVENT_LBUTTONUP :
        drawingp = False
    if event == cv2.EVENT_RBUTTONDOWN:
        drawingm = True

    elif event == cv2.EVENT_MOUSEMOVE :
        if drawingm == True :
            cv2.circle(img, (x, y), 3, (255, 0, 0), 4)
    elif event == cv2.EVENT_RBUTTONUP :
        drawingm = False

# Create a black image
img = np.zeros((512, 512, 3), np.uint8)
# Create a window and bind the function to window
cv2.namedWindow("Curve Window")
# Connect the mouse button to our callback function
cv2.setMouseCallback("Curve Window", draw_curve)
# display the window
while True:
    cv2.imshow("Curve Window", img)
    if cv2.waitKey(10) == 27:
        break

cv2.destroyAllWindows()
```


3.2 De l'interpolation guidé avec l'équation de Poisson.

Dans cette partie nous allons nous placer sur des matrices incarnant des images. Nous reprendrons pour partie les notations des sections précédentes. Afin de simplifier l'exposition nous nous placerons dans le cas d'une matrice i.e. d'une image noir et blanc, le cas de la couleur revenant à refaire les calculs sur les trois couches de l'image. Le but de cette partie est de faire de l'édition d'image : nous allons essayer de modifier une image en utilisant le contenu d'une autre image et/ou plus généralement un « champ de vecteur » donné à l'avance. Avant de travailler sur la matrice/image directement nous allons devoir faire un détour par une étape de formalisation de notre problème. Dans cette partie nous allons penser à des images U et F comme des fonctions à deux variables réelles définies sur le rectangle $]0, m+1[\times]0, n+1[$ à valeurs dans $\mathbb{R}$. **Dans toute cette partie on supposera que les fonctions intervenant sont de classe $\mathcal{C}^2$ et que F représente l'image à modifier.**

Soit $\Omega \subset]0, m+1[\times]0, n+1[$ un fermé connexe (i.e. d'un seul tenant) et notons $\partial\Omega$ la frontière ($:=$ le bord) de Ω . Moralement on considérera l'intérieur de Ω noté $\mathring{\Omega} := \Omega \setminus \partial\Omega$ comme un ensemble de pixels à modifier et le bord $\partial\Omega$ comme un ensemble de pixels à conserver. Le but est donc de trouver une fonction de classe $\mathcal{C}^2$:

$$u :]0, m+1[\times]0, n+1[\rightarrow \mathbb{R}$$

vérifiant certains critères, critères que nous développerons dans la suite, telle que la restriction de u au bord $\partial\Omega$ soit égale à une fonction f fixée sur ce même bord. La fonction f sur $\partial\Omega$ représente les pixels à conserver. On note cela :

$$u|_{\partial\Omega} = f|_{\partial\Omega}$$

On peut schématiser cette situation par le graphe suivant :

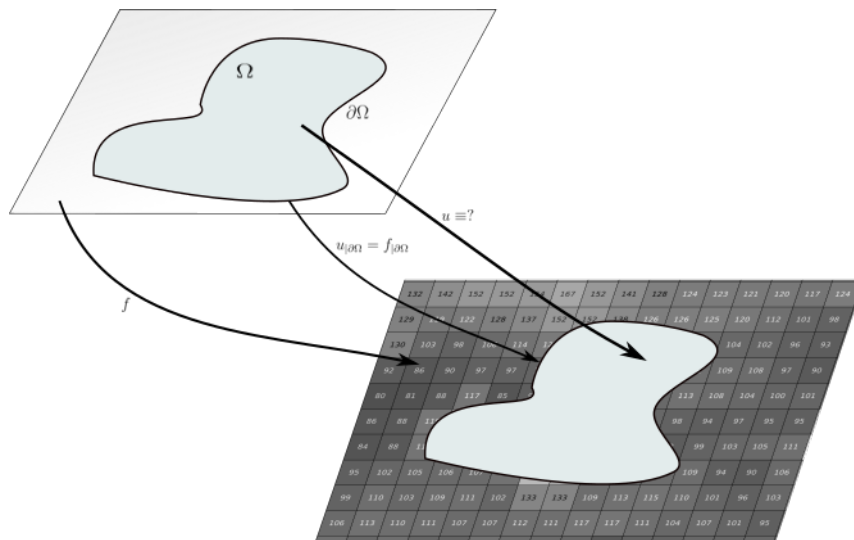


FIGURE 4 – La fonction f est bien définie sur l'extérieur de Ω et on cherche u tel que $u|_{\partial\Omega} = f|_{\partial\Omega}$ et u sur $\mathring{\Omega}$.

Afin de traiter ce problème nous allons choisir comme critère de complétion/d'interpolation pour la fonction u que les variations (=le gradient) de l'image à modifier doit suivre un champ de vecteur donné dans Ω tout en conservant les pixels donnés par f sur le bord. Mathématiquement nous allons chercher u telle que l'action suivante soit minimale sur Ω :

$$S(u) := \frac{1}{2} \iint_{\Omega} \|\nabla u(x, y) - g(x, y)\|_2^2 dx dy$$

où le $\frac{1}{2}$ et la norme au carré n'ont été rajouté que pour faciliter les calculs et $g : (g_x, g_y) : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ est un champ de vecteur de $\mathbb{R}^2$.

1. En utilisant la partie 1, montrer qu'un point critique u de S vérifie l'équation suivante :

$$(\mathcal{I}) \quad \begin{cases} \Delta u = \operatorname{div} g & \text{sur } \mathring{\Omega} \\ u = f & \text{sur } \partial\Omega \end{cases}$$

2. Comme pour la section précédente discrétisons le problème. Soit $U \in \mathcal{M}_{m,n}(\mathbb{R})$ la matrice qui représente la forme discrétisée de la fonction u précédente. On rappelle que cette matrice représente une image (en noir et blanc). De plus nous ferons l'abus de noter encore Ω la version discrétisée de Ω . Pour une image on supposera qu'un point $(x, y) \in \Omega$ est la position d'un pixel i.e x est le numéro de ligne de la matrice et y le numéro de la colonne. Afin de retrouver les notations « habituelles » en algèbre linéaire on notera plutôt : $(x, y) := (i, j)$ avec i, j deux entiers. Afin de modéliser l'ensemble Ω sur lequel nous allons modifier l'image, on introduit une matrice $M \in \mathcal{M}_{m,n}(\mathbb{R})$ composée uniquement de 0 et de 1. On appelle ce type de matrice **un masque**. Les coefficients de M vérifient donc :

$$M_{i,j} := \begin{cases} 1 & \text{si } (i, j) \in \mathring{\Omega} \\ 0 & \text{sinon} \end{cases}$$

Nous dirons qu'une matrice $A \in \mathcal{M}_{m,n}(\mathbb{R})$ est masquée (ohé, ohé) par le masque M si elle vérifie l'équation :

$$A = A \star M$$

où $\star$ note le produit d'Hadamard entre deux matrices¹. L'équation précédente signifie simplement que les coefficients $A_{i,j}$ de A vérifient l'implication suivante : si $M_{i,j} = 0$ alors $A_{i,j} = 0$.

- (a) Justifier rapidement que l'ensemble des matrices masquées (ohé, ohé) par le masque M :

$$E := \{A \in \mathcal{M}_{m,n}(\mathbb{R}) \mid A = A \star M\}$$

est un espace vectoriel muni d'un produit scalaire par la formule suivante :

$$\langle \bullet, \bullet \rangle = \text{Trace}(\bullet^T \bullet)$$

- (b) On considérera le pas de discrétisation $h = \Delta x = \Delta y = 1$ pour les images, en effet pour une image l'élément de longueur le plus petit possible est le passage d'un pixel à un autre i.e d'un entier à un autre. En utilisant la section précédente montrer qu'une approximation à l'ordre 2 du problème précédent $(\mathcal{I})$ s'écrit sous la forme :

$$(\mathcal{I}_d) \quad (L_m U + U L_n) \star M = (D_m G_x + G_y D_n^T) \star M$$

où L_m, L_n, D_m et D_n sont des matrices à déterminer et G_x et G_y sont les formes discrétisées (i.e. matricielles) de g_x et g_y respectivement.

- (c) Justifier alors que le problème précédent se résout grâce à l'algorithme du gradient conjugué de la partie 2.
(d) Supposons que $g = \nabla \phi$ où ϕ est une fonction de classe $\mathcal{C}^2$ de $\mathbb{R}^2$ dans $\mathbb{R}$. Montrer que le problème $(\mathcal{I})$ s'écrit alors :

$$(\mathcal{I}_{\nabla}) \quad \begin{cases} \Delta u = \Delta \phi & \text{sur } \mathring{\Omega} \\ u = f & \text{sur } \partial \Omega \end{cases}$$

et que l'on peut obtenir une forme discrétisée suivante :

$$(\mathcal{I}_{\nabla d}) \quad (L_m U + U L_n) \star M = (L_m \Phi + \Phi L_n) \star M$$

où Φ représente la forme discrétisée matricielle de ϕ .

3. **Application 1 :** Lorsqu'une image est « abîmée » sur une zone $\mathring{\Omega}$ de celle-ci, une idée pour la « restaurer » est de trouver u tel que le gradient de u sur $\mathring{\Omega}$ prolonge au mieux le gradient de l'image d'origine sur le bord $\partial \Omega$ de Ω . Pour le dire autrement, l'idée revient à compléter les endroits abîmés de l'image en suivant le gradient des parties non abîmées. En notant f la fonction modélisant l'image à restaurer, formellement on essaie de minimiser l'action suivante :

$$S(u) := \frac{1}{2} \iint_{\Omega} \|\nabla u(x, y) + \nabla \bar{f}(x, y)\|_2^2 dx dy$$

où $\bar{f}$ est la fonction définie par :

$$\bar{f}(x, y) := \begin{cases} f(x, y) & \text{si } (x, y) \in \partial \Omega \\ 0 & \text{si } (x, y) \in \mathring{\Omega} \end{cases}$$

En utilisant la question précédente, on voit que cela revient à résoudre le problème discrétisé suivant :

$$(\mathcal{I}_{\text{restauration}}) \quad (L_m U + U L_n) \star M = -(L_m (F \star (1_{m,n} - M)) + (F \star (1_{m,n} - M)) L_n) \star M$$

où F est la matrice de l'image « abîmée » et $1_{m,n}$ est la matrice de taille (m, n) ne contenant que des 1.

1. On rappelle que le produit d'Hadamard entre deux matrices est le produit terme à terme des coefficients des matrices.

- (a) Créer en utilisant OpenCV une fonction **draw_mask** qui prend en entrée une image F à restaurer et renvoie la matrice de masque M . On pourra utiliser le code donné dans la partie précédente. **La création de cette matrice devra se faire à l'aide de la souris directement sur une copie de l'image F .**



FIGURE 5 – à gauche : l'image originale, à droite : l'image avec le tracé du masque. La fonction **draw_mask** renvoie une matrice M de même taille que l'image originale ne contenant que des 0 (sur la partie extérieure au masque) et des 1 (sur le masque i.e. sur les tracés rouges sur l'image de droite).

- (b) En utilisant les parties précédentes créer une fonction **image_restoration** qui résout le problème ($\mathcal{I}_{\text{restoration}}$) à l'aide du gradient conjugué matriciel de la partie 2. N'oubliez pas les conversions des images en « float » avant les calculs et à la fin des « float » en « uint8 » afin de retrouver une image au bon format/type. De même il n'est pas nécessaire de prendre une tolérance ϵ trop petite ($\epsilon = 1$ suffit largement) pour les images de type « uint8 ». Pour éviter des valeurs aberrantes dans les calculs on pourra rajouter dans la boucle du gradient conjugué matriciel (à la fin de chaque boucle) :

```
X=np.maximum(0, np.minimum(X,255))
```

qui permet de normaliser les valeurs entre 0 et 255.

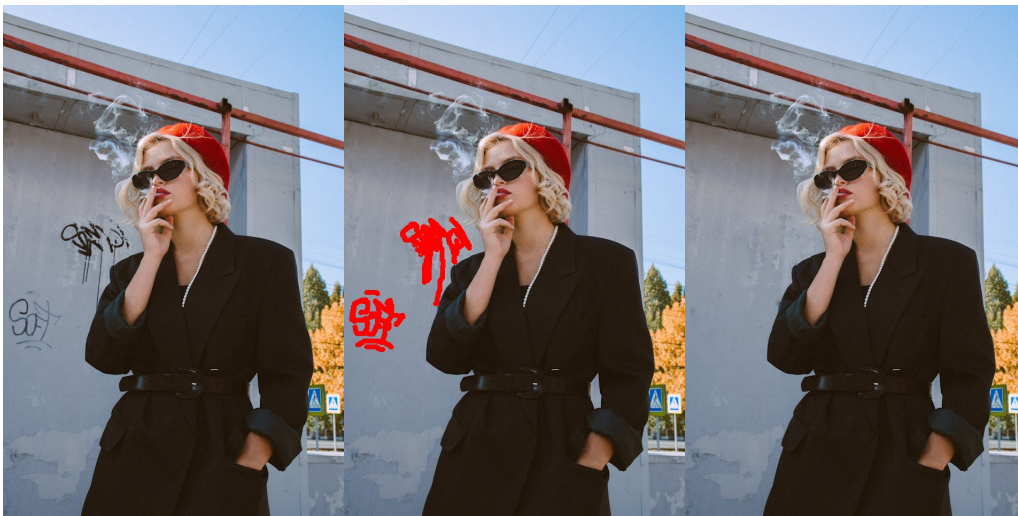


FIGURE 6 – à gauche : l'image originale, au centre : la création du masque, à droite le résultat final de l'algorithme.

- (c) Améliorer l'algorithme précédent en limitant l'application de l'algorithme du gradient conjugué à une sous-image contenant la partie où est appliquée le masque (penser à prendre quelques pixels de marges).
- (d) Tester sur plusieurs fonctions et masques votre fonction. Quels sont les limites de cet algorithme ?

4. **Application 2 :** L'application précédente nous a permis de faire « diffuser » des couleurs au bord du masque dans la partie « masquée » en suivant le gradient de notre image tout en imposant que le gradient de la fonction u soit le plus petit possible. Comme le montre la figure précédente, on voit que cela nous a permis de « supprimer » les graffitis. Néanmoins une des limites de l'algorithme précédent est que la diffusion à l'intérieur du masque n'est contrainte que par les couleurs au bord. Dans cette question nous allons formaliser le fait d'imposer le gradient le long duquel à lieu la diffusion à l'intérieur du masque. Nous reprenons ici les notations de la question précédente et introduisons une nouvelle image que l'on modélisera par une fonction g de classe $\mathcal{C}^2$ définie sur Ω à valeurs réelles. Nous allons alors imposer que la complétion de l'image f à l'intérieur de masque i.e. dans $\mathring{\Omega}$, la diffusion des couleurs s'appuie sur un mélange de ∇g le gradient de g et de ∇f gradient de f . D'un point de vue fonctionnel cela revient à poser l'action suivante :

$$S(u) := \frac{1}{2} \iint_{\Omega} \|\nabla u(x, y) + \nabla \bar{f}(x, y) - (\alpha \nabla g(x, y) + (1 - \alpha) \nabla f(x, y))\|_2^2 dx dy$$

où α est un réel tel que : $\alpha \in [0, 1]$. Notons $\mathcal{A}$ l'application matricielle définie par :

$$\mathcal{A}(U) := (L_m U + U L_n) \star M$$

En utilisant la question précédente, cela revient à résoudre la problème discrétisé suivant :

$$(\mathcal{I}_{\text{diff}}) \quad \mathcal{A}(U) = -\mathcal{A}(F \star (1_{m,n} - M)) + \alpha \mathcal{A}(G) + (1 - \alpha) \mathcal{A}(F)$$

où G est la matrice (l'image) associée à l'application g .

- (a) En utilisant les questions précédentes et à l'aide de la librairie Opencv créer une fonction qui permet en utilisant la souris de sélectionner une zone (notre futur masque) sur une image F . On pourra utiliser la fonction `cv2.fillPoly` qui permet de générer un polygone plein à partir d'une suite de point.



FIGURE 7 – **à gauche** : l'image originale, **à droite** : l'image avec le tracé du masque polygonal. Votre fonction pourra renvoyer une matrice M de même taille que l'image originale (ou bien une sous-image contenant la partie masquée) ne contenant que des 0 (sur la partie extérieure au masque) et des 1 (sur le masque i.e. sur les tracés rouges sur l'image de droite).

- (b) Créer alors un programme résolvant le problème $(\mathcal{I}_{\text{diff}})$ en utilisant le gradient conjugué matriciel. Votre programme :
- prendra en entrée une image F à modifier et une image G nous servant à la modification, ainsi qu'un $\alpha \in [0, 1]$ fixé.
 - En utilisant la question précédente sélectionner une zone à modifier sur l'image F (le masque de F) et une partie de l'image G nous servant de guide pour la diffusion le long du gradient de g .

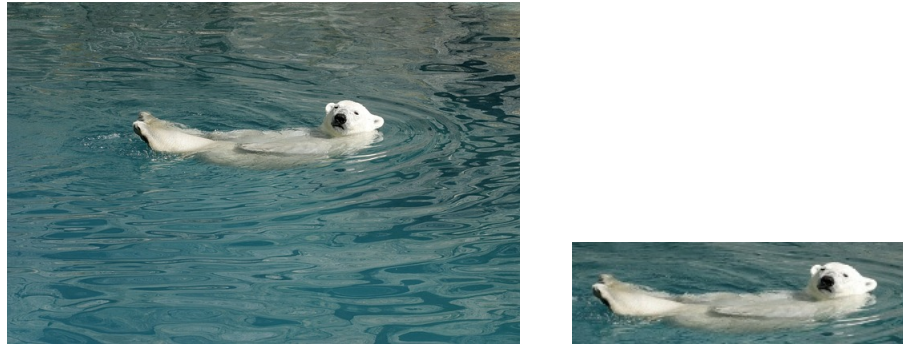


FIGURE 8 – à gauche : l'image G choisi, à droite : la partie de G sélectionnée que nous allons faire « diffuser » grâce à son gradient dans la partie en rouge de l'image originale.

- Enfin résolvez le problème $(\mathcal{I}_{\text{diff}})$ en utilisant le gradient conjugué matriciel et afficher l'image originale modifiée par l'image issue de G . Pour l'efficacité de l'algorithme il est conseillé de travailler sur une sous-image de F contenant la partie masquée (plus quelques pixels de marges...). Penser à redimensionner l'image extraite de G (la « petite » photographie à droite précédente) à la même dimension que l'image extraite de F .



FIGURE 9 – à gauche : l'image originale/source F , à droite : l'image G que nous allons utiliser pour compléter le masque, en bas : le résultat final de l'algorithme.

5. **Ouverture** : Les algorithmes précédents admettent de nombreuses variations. En vous inspirant de l'article suivant : [lien](#), réaliser certaines de ces variations possibles. Vos résultats mêmes partiels si non aboutis doivent apparaître dans le rapport et seront commentés afin d'analyser les points qui posent problèmes.